SNES Assembly Adventure 07: Direct Memory Access and Bouncing Sprites

30 May 2019

Update February 2022: All code examples from all articles in this series can now be found on Github in one repository

Welcome back, Adventurer! Last time, we learned a bit more about subroutines and how to pass arguments to them.

This time, we'll see this in more concrete action and

- revisit the sprite demo from part 4
- make the code more reusable by replacing some sections with subroutines
- move data from WRAM to OAMRAM with DMA, or *Direct Memory Access*
- make the sprites move and bounce off the screen boundaries

So lots to get to. Let's get started!



I forgot the Subroutine Badge, you'll be awarded it next time

Sprite Demo Revisited

Let's look at the sprite demo code again:

```
;----- Aliases/Labels -----------------------------------------------------
; these are aliases for the Memory Mapped Registers we will use
INIDISP     = $2100     ; inital settings for screen
OBJSEL      = $2101     ; object size $ object data area designation
OAMADDL     = $2102     ; address for accessing OAM
OAMADDH     = $2103
OAMDATA     = $2104     ; data for OAM write
VMAINC      = $2115     ; VRAM address increment value designation
VMADDL      = $2116     ; address for VRAM read and write
VMADDH      = $2117
VMDATAL     = $2118     ; data for VRAM write
VMDATAH     = $2119     ; data for VRAM write
CGADD       = $2121     ; address for CGRAM read and write
CGDATA      = $2122     ; data for CGRAM write
TM          = $212c     ; main screen designation
NMITIMEN    = $4200     ; enable flaog for v-blank
```

```
17  RDNMI       = $4210     ; read the NMI flag status
18  ;-------------------------------------------------------------------------
19
20  ;----- Assembler Directives ----------------------------------------------
21  .p816                         ; tell the assembler this is 65816 code
22  ;-------------------------------------------------------------------------
23
24  ;----- Includes ----------------------------------------------------------
25  .segment "SPRITEDATA"
26  SpriteData: .incbin "Sprites.vra"
27  ColorData:  .incbin "SpriteColors.pal"
28  ;-------------------------------------------------------------------------
29
30  .segment "CODE"
31  ;-------------------------------------------------------------------------
32  ;   This is the entry point of the demo
33  ;-------------------------------------------------------------------------
34  .proc   ResetHandler
35          sei                   ; disable interrupts
36          clc                   ; clear the carry flag
37          xce                   ; switch the 65816 to native (16-bit mode)
38          lda #$8f              ; force v-blanking
39          sta INIDISP
40          stz NMITIMEN          ; disable NMI
41
42          ; transfer VRAM data
43          stz VMADDL            ; set the VRAM address to $0000
44          stz VMADDH
45          lda #$80
46          sta VMAINC            ; increment VRAM address by 1 when writing to
47          ldx #$00              ; set register X to zero, we will use X as a l
48  VRAMLoop:
49          lda SpriteData, X     ; get bitplane 0/2 byte from the sprite data
50          sta VMDATAL           ; write the byte in A to VRAM
51          inx                   ; increment counter/offset
52          lda SpriteData, X     ; get bitplane 1/3 byte from the sprite data
53          sta VMDATAH           ; write the byte in A to VRAM
54          inx                   ; increment counter/offset
55          cpx #$80              ; check whether we have written $04 * $20 = $8
56          bcc VRAMLoop          ; if X is smaller than $80, continue the loop
57
58          ; transfer CGRAM data
59          lda #$80
60          sta CGADD             ; set CGRAM address to $80
```

```
61          ldx #$00                ; set X to zero, use it as loop counter and of
62  CGRAMLoop:
63          lda ColorData, X        ; get the color low byte
64          sta CGDATA              ; store it in CGRAM
65          inx                     ; increase counter/offset
66          lda ColorData, X        ; get the color high byte
67          sta CGDATA              ; store it in CGRAM
68          inx                     ; increase counter/offset
69          cpx #$20                ; check whether 32/$20 bytes were transfered
70          bcc CGRAMLoop           ; if not, continue loop
71
72          .byte $42, $00          ; debugger breakpoint
73
74          ; set up OAM data
75          stz OAMADDL             ; set the OAM address to ...
76          stz OAMADDH             ; ...at $0000
77          ; OAM data for first sprite
78          lda # (256/2 - 8)       ; horizontal position of first sprite
79          sta OAMDATA
80          lda # (224/2 - 8)       ; vertical position of first sprite
81          sta OAMDATA
82          lda #$00                ; name of first sprite
83          sta OAMDATA
84          lda #$00                ; no flip, prio 0, palette 0
85          sta OAMDATA
86          ; OAM data for second sprite
87          lda # (256/2)           ; horizontal position of second sprite
88          sta OAMDATA
89          lda # (224/2 - 8)       ; vertical position of second sprite
90          sta OAMDATA
91          lda #$01                ; name of second sprite
92          sta OAMDATA
93          lda #$00                ; no flip, prio 0, palette 0
94          sta OAMDATA
95          ; OAM data for third sprite
96          lda # (256/2 - 8)       ; horizontal position of third sprite
97          sta OAMDATA
98          lda # (224/2)           ; vertical position of third sprite
99          sta OAMDATA
100         lda #$02                ; name of third sprite
101         sta OAMDATA
102         lda #$00                ; no flip, prio 0, palette 0
103         sta OAMDATA
104         ; OAM data for fourth sprite
```

```
105         lda # (256/2)           ; horizontal position of fourth sprite
106         sta OAMDATA
107         lda # (224/2)           ; vertical position of fourth sprite
108         sta OAMDATA
109         lda #$03                ; name of fourth sprite
110         sta OAMDATA
111         lda #$00                ; no flip, prio 0, palette 0
112         sta OAMDATA
113
114         ; make Objects visible
115         lda #$10
116         sta TM
117         ; release forced blanking, set screen to full brightness
118         lda #$0f
119         sta INIDISP
120         ; enable NMI, turn on automatic joypad polling
121         lda #$81
122         sta NMITIMEN
123
124         jmp GameLoop            ; all initialization is done
125 .endproc
126 ;---------------------------------------------------------------------------
127
128 ;---------------------------------------------------------------------------
129 ;   After the ResetHandler will jump to here
130 ;---------------------------------------------------------------------------
131 ; .smart ; keep track of registers widths
132 .proc   GameLoop
133         wai                     ; wait for NMI / V-Blank
134
135         ; here we would place all of the game logic
136         ; and loop forever
137
138         jmp GameLoop
139 .endproc
140 ;---------------------------------------------------------------------------
141
142 ;---------------------------------------------------------------------------
143 ;   Will be called during V-Blank
144 ;---------------------------------------------------------------------------
145 .proc   NMIHandler
146         lda RDNMI               ; read NMI status, acknowledge NMI
147
148         ; this is where we would do graphics update
```

```
149
150         rti
151 .endproc
152 ;--------------------------------------------------------------------------
153
154 ;--------------------------------------------------------------------------
155 ;   Is not used in this program
156 ;--------------------------------------------------------------------------
157 .proc   IRQHandler
158         ; code
159         rti
160 .endproc
161 ;--------------------------------------------------------------------------
162
163 ;--------------------------------------------------------------------------
164 ;   Interrupt and Reset vectors for the 65816 CPU
165 ;--------------------------------------------------------------------------
166 .segment "VECTOR"
167 ; native mode   COP,        BRK,        ABT,
168 .addr           $0000,      $0000,      $0000
169 ;               NMI,        RST,        IRQ
170 .addr           NMIHandler, $0000,      $0000
171
172 .word           $0000, $0000    ; four unused bytes
173
174 ; emulation m.  COP,        BRK,        ABT,
175 .addr           $0000,      $0000,      $0000
176 ;               NMI,        RST,        IRQ
177 .addr           $0000,      ResetHandler, $0000
```

SpriteDemo.asm hosted with ❤ by **GitHub** **view raw**

If we scan the code closely, we see that there is a lot of code repetition. For example, the two loops `VRAMLoop` and `CGRAMLoop` starting at lines 42 and 58 respectively are almost identical. The same goes for the code between lines 74 and 112 that set up the OAMRAM data for all four sprites. If you have programmed in any other language before, you'd probably immediately go for a function to keep your code shorter, more readable, and concise.

We will replace those two sections mentioned above with subroutines. These subroutines will be very useful later on when we write more complicated demos and games. Moving data to VRAM, CGRAM, and OAMRAM is something we do pretty much *every* frame.

So let's start by moving some of the repetitive code into subroutines:

```
;----- Aliases/Labels -----------------------------------------------------
; these are aliases for the Memory Mapped Registers we will use
INIDISP     = $2100     ; inital settings for screen
OBJSEL      = $2101     ; object size $ object data area designation
OAMADDL     = $2102     ; address for accessing OAM
OAMADDH     = $2103
OAMDATA     = $2104     ; data for OAM write
VMAINC      = $2115     ; VRAM address increment value designation
VMADDL      = $2116     ; address for VRAM read and write
VMADDH      = $2117
VMDATAL     = $2118     ; data for VRAM write
VMDATAH     = $2119
CGADD       = $2121     ; address for CGRAM read and write
CGDATA      = $2122     ; data for CGRAM write
TM          = $212c     ; main screen designation
NMITIMEN    = $4200     ; enable flaog for v-blank
RDNMI       = $4210     ; read the NMI flag status
;--------------------------------------------------------------------------

;----- Assembler Directives -----------------------------------------------
.p816                           ; tell the assembler this is 65816 code
;--------------------------------------------------------------------------

;----- Includes -----------------------------------------------------------
.segment "SPRITEDATA"
SpriteData: .incbin "Sprites.vra"
ColorData:  .incbin "SpriteColors.pal"
;--------------------------------------------------------------------------

.segment "CODE"
;--------------------------------------------------------------------------
;   This is the entry point of the demo
;--------------------------------------------------------------------------
.proc   ResetHandler
        sei                     ; disable interrupts
        clc                     ; clear the carry flag
        xce                     ; switch the 65816 to native (16-bit mode)
        rep #$10                ; clear the Index Register Select flag
        sep #$20                ; set the Memory/Accumulator Select flag
        lda #$8f                ; force v-blanking
        sta INIDISP
        stz NMITIMEN            ; disable NMI
        ldx #$1fff              ; set stack pointer to $1fff
        txs
```

```
45
46          jsr LoadVRAM            ; transfer VRAM data
47          jsr LoadCGRAM           ; transfer color data into CGRAM
48          jsr LoadOAMRAM          ; transfer sprite data to OAMRAM
49
50          .byte $42, $00          ; debugger breakpoint
51
52          ; make Objects visible
53          lda #$10
54          sta TM
55          ; release forced blanking, set screen to full brightness
56          lda #$0f
57          sta INIDISP
58          ; enable NMI, turn on automatic joypad polling
59          lda #$81
60          sta NMITIMEN
61
62          jmp GameLoop            ; all initialization is done
63  .endproc
64  ;-------------------------------------------------------------------------------
65
66  ;-------------------------------------------------------------------------------
67  ;   After the ResetHandler will jump to here
68  ;-------------------------------------------------------------------------------
69  ; .smart ; keep track of registers widths
70  .proc   GameLoop
71          wai                     ; wait for NMI / V-Blank
72
73          ; here we would place all of the game logic
74          ; and loop forever
75
76          jmp GameLoop
77  .endproc
78  ;-------------------------------------------------------------------------------
79
80  ;-------------------------------------------------------------------------------
81  ;   Will be called during V-Blank
82  ;-------------------------------------------------------------------------------
83  .proc   NMIHandler
84          lda RDNMI               ; read NMI status, acknowledge NMI
85
86          ; this is where we would do graphics update
87
88          rti
```

```
.endproc
;---------------------------------------------------------------------------

;---------------------------------------------------------------------------
;   Is not used in this program
;---------------------------------------------------------------------------
.proc   IRQHandler
        ; code
        rti
.endproc
;---------------------------------------------------------------------------

;---------------------------------------------------------------------------
;   Load sprite data into VRAM
;---------------------------------------------------------------------------
.proc   LoadVRAM
        ; transfer VRAM data
        stz VMADDL              ; set the VRAM address to $0000
        stz VMADDH
        lda #$80
        sta VMAINC              ; increment VRAM address by 1 when writing to
        ldx #$00                ; set register X to zero, we will use X as a l
VRAMLoop:
        lda SpriteData, X       ; get bitplane 0/2 byte from the sprite data
        sta VMDATAL             ; write the byte in A to VRAM
        inx                     ; increment counter/offset
        lda SpriteData, X       ; get bitplane 1/3 byte from the sprite data
        sta VMDATAH             ; write the byte in A to VRAM
        inx                     ; increment counter/offset
        cpx #$80                ; check whether we have written $04 * $20 = $8
        bcc VRAMLoop            ; if X is smaller than $80, continue the loop

        rts
.endproc
;---------------------------------------------------------------------------

;---------------------------------------------------------------------------
;   Load color data into CGRAM
;---------------------------------------------------------------------------
.proc   LoadCGRAM
        ; transfer CGRAM data
        lda #$80
        sta CGADD               ; set CGRAM address to $80
        ldx #$00                ; set X to zero, use it as loop counter and of
```

```
133 CGRAMLoop:
134         lda ColorData, X        ; get the color low byte
135         sta CGDATA              ; store it in CGRAM
136         inx                     ; increase counter/offset
137         lda ColorData, X        ; get the color high byte
138         sta CGDATA              ; store it in CGRAM
139         inx                     ; increase counter/offset
140         cpx #$20                ; check whether 32/$20 bytes were transfered
141         bcc CGRAMLoop           ; if not, continue loop
142
143         rts
144 .endproc
145 ;---------------------------------------------------------------------------
146
147 ;---------------------------------------------------------------------------
148 ;   Load sprite informations into OAMRAM
149 ;---------------------------------------------------------------------------
150 .proc   LoadOAMRAM
151         ; set up OAM data
152         stz OAMADDL             ; write to OAMRAM starting at address $0000
153         stz OAMADDH
154         ; OAM data for first sprite
155         lda # (256/2 - 8)       ; horizontal position of first sprite
156         sta OAMDATA
157         lda # (224/2 - 8)       ; vertical position of first sprite
158         sta OAMDATA
159         lda #$00                ; name of first sprite
160         sta OAMDATA
161         lda #$00                ; no flip, prio 0, palette 0
162         sta OAMDATA
163         ; OAM data for second sprite
164         lda # (256/2)           ; horizontal position of second sprite
165         sta OAMDATA
166         lda # (224/2 - 8)       ; vertical position of second sprite
167         sta OAMDATA
168         lda #$01                ; name of second sprite
169         sta OAMDATA
170         lda #$00                ; no flip, prio 0, palette 0
171         sta OAMDATA
172         ; OAM data for third sprite
173         lda # (256/2 - 8)       ; horizontal position of third sprite
174         sta OAMDATA
175         lda # (224/2)           ; vertical position of third sprite
176         sta OAMDATA
```

```
177           lda #$02                ; name of third sprite
178           sta OAMDATA
179           lda #$00                ; no flip, prio 0, palette 0
180           sta OAMDATA
181           ; OAM data for fourth sprite
182           lda # (256/2)           ; horizontal position of fourth sprite
183           sta OAMDATA
184           lda # (224/2)           ; vertical position of fourth sprite
185           sta OAMDATA
186           lda #$03                ; name of fourth sprite
187           sta OAMDATA
188           lda #$00                ; no flip, prio 0, palette 0
189           sta OAMDATA
190
191           rts
192   .endproc
193   ;--------------------------------------------------------------------------
194
195   ;--------------------------------------------------------------------------
196   ;   Interrupt and Reset vectors for the 65816 CPU
197   ;--------------------------------------------------------------------------
198   .segment "VECTOR"
199   ; native mode   COP,         BRK,         ABT,
200   .addr           $0000,       $0000,       $0000
201   ;               NMI,         RST,         IRQ
202   .addr           NMIHandler, $0000,       $0000
203
204   .word           $0000, $0000    ; four unused bytes
205
206   ; emulation m.  COP,         BRK,         ABT,
207   .addr           $0000,       $0000,       $0000
208   ;               NMI,         RST,         IRQ
209   .addr           $0000,       ResetHandler, $0000
```

SpriteDemo2.asm hosted with ❤ by **GitHub** **view raw**

This code shouldn't hold any surprises for you by now. We simply moved the code for loading VRAM, CGRAM, and OAMRAM into subroutines and replaced the moved code sections with subroutine calls. Easy.

But there's still room for improvement. In `LoadVRAM`, we use the label `SpriteData` to access the sprites' raw data in memory. While this works, what if you want to load memory from

another memory location than `SpriteData`? You'd need to rewrite the whole subroutine. Not very convenient.

But thankfully, there's a solution.

Passing Addresses as Arguments

Last time, we learned how to pass arguments to subroutines. We'll use this now to improve `LoadVRAM`. So, what information does the subroutine needs to perform its task? In essence, it needs three bits (*no pun intended*) of information:

- The number of bytes to be transferred to VRAM
- The source address where the sprite data is stored
- The VRAM starting address

We'll pass the arguments by stack. Let's see this in action, here's the updated code:

```
1  ;----- Aliases/Labels -----------------------------------------------------
2  ; these are aliases for the Memory Mapped Registers we will use
3  INIDISP     = $2100     ; inital settings for screen
4  OBJSEL      = $2101     ; object size $ object data area designation
5  OAMADDL     = $2102     ; address for accessing OAM
6  OAMADDH     = $2103
7  OAMDATA     = $2104     ; data for OAM write
8  VMAINC      = $2115     ; VRAM address increment value designation
9  VMADDL      = $2116     ; address for VRAM read and write
10 VMADDH      = $2117
11 VMDATAL     = $2118     ; data for VRAM write
12 VMDATAH     = $2119
13 CGADD       = $2121     ; address for CGRAM read and write
14 CGDATA      = $2122     ; data for CGRAM write
15 TM          = $212c     ; main screen designation
16 NMITIMEN    = $4200     ; enable flaog for v-blank
17 RDNMI       = $4210     ; read the NMI flag status
18 ;--------------------------------------------------------------------------
19
20 ;----- Assembler Directives ---------------------------------------------
21 .p816                         ; tell the assembler this is 65816 code
22 ;--------------------------------------------------------------------------
23
24 ;----- Includes ----------------------------------------------------------
25 .segment "SPRITEDATA"
26 SpriteData: .incbin "Sprites.vra"
```

```
27  ColorData:  .incbin "SpriteColors.pal"
28  ;------------------------------------------------------------------------
29
30  .segment "CODE"
31  ;------------------------------------------------------------------------
32  ;   This is the entry point of the demo
33  ;------------------------------------------------------------------------
34  .proc   ResetHandler
35          sei                     ; disable interrupts
36          clc                     ; clear the carry flag
37          xce                     ; switch the 65816 to native (16-bit mode)
38          rep #$10                ; set X and Y to 16-bit
39          sep #$20                ; set A to 8-bit
40          lda #$8f                ; force v-blanking
41          sta INIDISP
42          stz NMITIMEN            ; disable NMI
43          ; set the stack pointer to $1fff
44          ldx #$1fff              ; load X with $1fff
45          txs                     ; copy X to stack pointer
46
47          ; load sprites into VRAM
48          tsx                     ; save current stack pointer
49          pea $0000               ; push VRAM destination address to stack
50          pea SpriteData          ; push sprite source address to stack
51          lda #$80                ; push number of bytes (128/$80) to transfer t
52          pha
53          jsr LoadVRAM            ; transfer VRAM data
54          txs                     ; "delete" data on stack by restoring old stac
55
56          ; load color data into CGRAM
57          tsx                     ; save current stack pointer
58          lda #$80                ; push CGRAM destination address to stack
59          pha
60          pea ColorData           ; push color source address to stack
61          lda #$20                ; push number of bytes (32/$20) to transfer to
62          pha
63          jsr LoadCGRAM           ; transfer color data into CGRAM
64          txs                     ; "delete" data on stack by restoring old stac
65
66          ; load sprite data into OAMRAM
67          jsr LoadOAMRAM          ; transfer sprite data to OAMRAM
68
69          .byte $42, $00          ; debugger breakpoint
70
```

```
71          ; make Objects visible
72          lda #$10
73          sta TM
74          ; release forced blanking, set screen to full brightness
75          lda #$0f
76          sta INIDISP
77          ; enable NMI, turn on automatic joypad polling
78          lda #$81
79          sta NMITIMEN
80
81          jmp GameLoop            ; all initialization is done
82  .endproc
83  ;-------------------------------------------------------------------------------
84
85  ;-------------------------------------------------------------------------------
86  ;   After the ResetHandler will jump to here
87  ;-------------------------------------------------------------------------------
88  ; .smart ; keep track of registers widths
89  .proc   GameLoop
90          wai                     ; wait for NMI / V-Blank
91
92          ; here we would place all of the game logic
93          ; and loop forever
94
95          jmp GameLoop
96  .endproc
97  ;-------------------------------------------------------------------------------
98
99  ;-------------------------------------------------------------------------------
100 ;   Will be called during V-Blank
101 ;-------------------------------------------------------------------------------
102 .proc   NMIHandler
103         lda RDNMI               ; read NMI status, acknowledge NMI
104
105         ; this is where we would do graphics update
106
107         rti
108 .endproc
109 ;-------------------------------------------------------------------------------
110
111 ;-------------------------------------------------------------------------------
112 ;   Is not used in this program
113 ;-------------------------------------------------------------------------------
114 .proc   IRQHandler
```

```
115         ; code
116         rti
117 .endproc
118 ;--------------------------------------------------------------------------
119
120 ;--------------------------------------------------------------------------
121 ;   Load sprite data into VRAM
122 ;   Parameters: NumBytes: .byte, SrcPointer: .addr, DestPointer: .addr
123 ;--------------------------------------------------------------------------
124 .proc   LoadVRAM
125         phx                     ; save old stack pointer
126         ; create frame pointer
127         phd                     ; push Direct Register to stack
128         tsc                     ; transfer Stack to... (via Accumulator)
129         tcd                     ; ...Direct Register.
130         ; use constants to access arguments on stack with Direct Addressing
131         NumBytes    = $07       ; number of bytes to transfer
132         SrcPointer  = $08       ; source address of sprite data
133         DestPointer = $0a       ; destination address in VRAM
134
135         ; set destination address in VRAM, and address increment after writing
136         ldx DestPointer         ; load the destination pointer...
137         stx VMADDL              ; ...and set VRAM address register to it
138         lda #$80
139         sta VMAINC              ; increment VRAM address by 1 when writing to
140
141         ; loop through source data and transfer to VRAM
142         ldy #$0000              ; set register Y to zero, we will use Y as a l
143 VRAMLoop:
144         lda (SrcPointer, S), Y  ; get bitplane 0/2 byte from the sprite data
145         sta VMDATAL             ; write the byte in A to VRAM
146         iny                     ; increment counter/offset
147         lda (SrcPointer, S), Y  ; get bitplane 1/3 byte from the sprite data
148         sta VMDATAH             ; write the byte in A to VRAM
149         iny                     ; increment counter/offset
150         cpy NumBytes            ; check whether we have written $04 * $20 = $8
151         bcc VRAMLoop            ; if X is smaller than $80, continue the loop
152
153         ; all done
154         pld                     ; restore caller's frame pointer
155         plx                     ; restore old stack pointer
156         rts
157 .endproc
158 ;--------------------------------------------------------------------------
```

```
159
160 ;------------------------------------------------------------------------
161 ;   Load color data into CGRAM
162 ;   NumBytes: .byte, SrcPointer: .byte, DestPointer: .addr
163 ;------------------------------------------------------------------------
164 .proc   LoadCGRAM
165         phx                     ; save old stack pointer
166         ; create frame pointer
167         phd                     ; push Direct Register to stack
168         tsc                     ; transfer Stack to... (via Accumulator)
169         tcd                     ; ...Direct Register.
170         ; use constants to access arguments on stack with Direct Addressing
171         NumBytes    = $07       ; number of bytes to transfer
172         SrcPointer  = $08       ; source address of sprite data
173         DestPointer = $0a       ; destination address in VRAM
174
175         ; set CGDRAM destination address
176         lda DestPointer         ; get destination address
177         sta CGADD               ; set CGRAM destination address
178
179         ldy #$0000              ; set Y to zero, use it as loop counter and of
180 CGRAMLoop:
181         lda (SrcPointer, S), Y  ; get the color low byte
182         sta CGDATA              ; store it in CGRAM
183         iny                     ; increase counter/offset
184         lda (SrcPointer, S), Y  ; get the color high byte
185         sta CGDATA              ; store it in CGRAM
186         iny                     ; increase counter/offset
187         cpy NumBytes            ; check whether 32/$20 bytes were transfered
188         bcc CGRAMLoop           ; if not, continue loop
189
190         ; all done
191         pld                     ; restore caller's frame pointer
192         plx                     ; restore old stack pointer
193         rts
194 .endproc
195 ;------------------------------------------------------------------------
196
197 ;------------------------------------------------------------------------
198 ;   Load sprite informations into OAMRAM
199 ;------------------------------------------------------------------------
200 .proc   LoadOAMRAM
201         ; set up OAM data
202         stz OAMADDL             ; write to OAMRAM starting at address $0000
```

```
203         stz OAMADDH
204         ; OAM data for first sprite
205         lda # (256/2 - 8)       ; horizontal position of first sprite
206         sta OAMDATA
207         lda # (224/2 - 8)       ; vertical position of first sprite
208         sta OAMDATA
209         lda #$00                ; name of first sprite
210         sta OAMDATA
211         lda #$00                ; no flip, prio 0, palette 0
212         sta OAMDATA
213         ; OAM data for second sprite
214         lda # (256/2)           ; horizontal position of second sprite
215         sta OAMDATA
216         lda # (224/2 - 8)       ; vertical position of second sprite
217         sta OAMDATA
218         lda #$01                ; name of second sprite
219         sta OAMDATA
220         lda #$00                ; no flip, prio 0, palette 0
221         sta OAMDATA
222         ; OAM data for third sprite
223         lda # (256/2 - 8)       ; horizontal position of third sprite
224         sta OAMDATA
225         lda # (224/2)           ; vertical position of third sprite
226         sta OAMDATA
227         lda #$02                ; name of third sprite
228         sta OAMDATA
229         lda #$00                ; no flip, prio 0, palette 0
230         sta OAMDATA
231         ; OAM data for fourth sprite
232         lda # (256/2)           ; horizontal position of fourth sprite
233         sta OAMDATA
234         lda # (224/2)           ; vertical position of fourth sprite
235         sta OAMDATA
236         lda #$03                ; name of fourth sprite
237         sta OAMDATA
238         lda #$00                ; no flip, prio 0, palette 0
239         sta OAMDATA
240
241         rts
242 .endproc
243 ;--------------------------------------------------------------------------
244
245 ;--------------------------------------------------------------------------
246 ;   Interrupt and Reset vectors for the 65816 CPU
```

```
247  ;--------------------------------------------------------------------------
248  .segment "VECTOR"
249  ; native mode   COP,        BRK,        ABT,
250  .addr           $0000,      $0000,      $0000
251  ;               NMI,        RST,        IRQ
252  .addr           NMIHandler, $0000,      $0000
253
254  .word           $0000, $0000    ; four unused bytes
255
256  ; emulation m.  COP,        BRK,        ABT,
257  .addr           $0000,      $0000,      $0000
258  ;               NMI,        RST,        IRQ
259  .addr           $0000,      ResetHandler, $0000
```

SpriteDemo3.asm hosted with ❤ by **GitHub** **view raw**

Here you see two updated code excerpts. Let's go through it line by line.

Lines 47 through 54: This should look really familiar to what we did last time when passing arguments by stack. But here is a new instruction:

```
pea : Push Effective Address
```

Push Effective Address or PEA pushes a 16-bit word onto the stack. Mind that PEA pushes the exact value onto the stack given to it as an operand. Some assemblers will allow `pea #$0000`, but most don't. I mention this because at times people get thrown off by PEA. `pea $1234` will push the bytes `$12` and `$34` onto the stack, *not* the byte at $1234. Also, PEA will *always* push two bytes to stack, so something like `pea $80` will not work. That's why we use LDA and PHA in lines 51 and 52 to push a single byte to stack.

So, we push first the VRAM starting address to stack, followed by the source address, and finally, the number of bytes we want to transfer to VRAM. No surprises here.

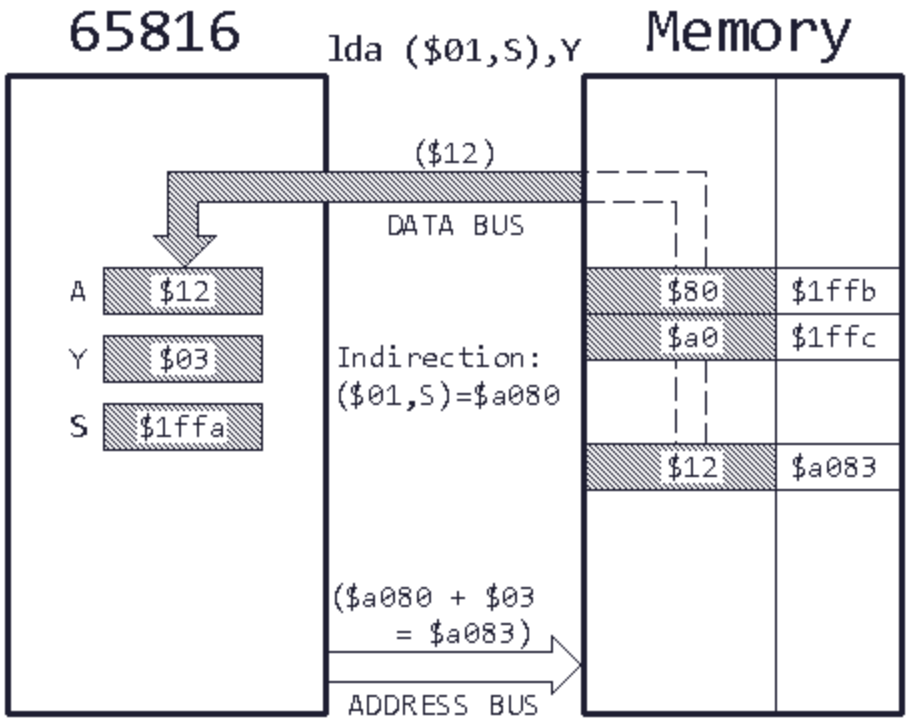
Lines 57 and 64 might look a bit odd. This is an optimization trick to save CPU cycles. If you look at the code from the article on subroutines and arguments, after the subroutine call, we had to "clean up" the stack by pulling back as many bytes as we pushed to stack before the subroutine call. This can be achieved more easily by simply storing the old stack pointer value in X before pushing all arguments to stack. Then, in the subroutine (line 125) we first save the old stack pointer with PHX on the stack. Then we execute the subroutine normally. Then, after the

subroutine, we simply pull the old stack pointer from the stack back into X and transfer it back to the stack pointer register (line 155).

This might look a bit weird at first. But the essential thing to understand is that the stack pointer has the exact same value before you push all arguments to stack, and after the subroutine has been called. This ensures you'll never run into problems due to stack over- or underflows or wrong return addresses.

Next, let's look at the subroutine itself.

Lines 125 through 157: This is essentially the same code as before. We add the creation of a frame pointer and local symbols to access the arguments on the stack. The interesting new thing here is found in lines 144 and 147. It's a new addressing mode! Let's look at *Stack Relative Indirect Indexed Addressing*:



Stack Relative Indirect Indexed Addressing

You know *Stack Relative Addressing*, and *Absolute Indexed Addressing* already. Think of it as a combination of the two. The effective address is the indirect address found on the stack, offset by a constant (the one given in the operand). Then, this base address is offset again by (the value in) Y.

Another way to look at it is to think of the address passed on stack as a pointer to the first element of an array. Then Y is the index to access a specific element in that array.

It might seem a bit convoluted yet, but once you get more experience with machine programming, you'll come to appreciate indirection and offsets in addressing. They make your code more flexible.

I'll additional information about *indirect addressing* in the *Links and References* section at the end of this article.

Lines 164 through 194: This shouldn't surprise you either, it's pretty much the same as in the `LoadVRAM` subroutine.

If you're struggling with *Stack Relative Indirect Indexed Addressing*, check out page 23 of the 65C816S datasheet.

The rest of the code hasn't changed so far. Now, if you think the code in `LoadOAMRAM` looks a bit ugly and inefficient, you are right. We'll now finally add some action by making the sprites move each frame and bounce off the screen boundaries.

Make the Sprites Move and Bounce

Let's finally add some action to this demo! Before we start, here are a few things to keep in mind to understand the code in this section:

1. We cannot access VRAM and OAMRAM directly, we have to use memory mapped registers to access and modify what is displayed on the screen
2. The OAMRAM holds the sprite name/index, color, and position data for each sprite
3. The `NMIHandler` subroutine is called each frame during V-blank

Here's how we're going to do it. Instead of setting data in OAMRAM "manually", as we did in `LoadOAMRAM`, we will designate a section in WRAM for OAM data and *copy the whole OAMRAM mirror section from WRAM into OAMRAM each frame*. For this, we will use a new technique called *Direct Memory Access*, or DMA. This might be a bit of a big leap, but I'll try my best to explain it in detail. I'll revisit DMA in the future several times.

This is a technique used in many SNES games. Since we can access WRAM directly without going through memory mapped registers first, it is easier for us to update the position, color, etc. of sprites this way and then copy it into OAMRAM in one go.

So we need to write three new sections of code:

- Designate a section of VRAM as our OAMRAM mirror
- Game logic code that updates the OAMRAM mirror; that is, move the sprites and check for collisions with the screen boundaries
- A subroutine that copies the OAMRAM mirror into OAMRAM each frame

So, let's dive into the code. As always, I show you the code first, then go through the changes line by line. Most of it is a copy of `SpriteDemo3.s` above, so I'll highlight the differences only.

```
1   ;----- Aliases/Labels -------------------------------------------------------
2   ; these are aliases for the Memory Mapped Registers we will use
3   INIDISP     = $2100     ; inital settings for screen
4   OBJSEL      = $2101     ; object size $ object data area designation
5   OAMADDL     = $2102     ; address for accessing OAM
6   OAMADDH     = $2103
7   OAMDATA     = $2104     ; data for OAM write
8   VMAINC      = $2115     ; VRAM address increment value designation
9   VMADDL      = $2116     ; address for VRAM read and write
10  VMADDH      = $2117
11  VMDATAL     = $2118     ; data for VRAM write
12  VMDATAH     = $2119
13  CGADD       = $2121     ; address for CGRAM read and write
14  CGDATA      = $2122     ; data for CGRAM write
15  TM          = $212c     ; main screen designation
16  NMITIMEN    = $4200     ; enable flaog for v-blank
17  MDMAEN      = $420b     ; DMA enable register
18  RDNMI       = $4210     ; read the NMI flag status
19  DMAP0       = $4300     ; DMA control register, channel 0
20  BBAD0       = $4301     ; DMA destination register, channel 0
21  A1T0L       = $4302     ; DMA source address register low, channel 0
22  A1T0H       = $4303     ; DMA source address register high, channel 0
23  A1T0B       = $4304     ; DMA source address register bank, channel 0
24  DAS0L       = $4305     ; DMA size register low, channel 0
25  DAS0H       = $4306     ; DMA size register high, channel 0
26  ;------------------------------------------------------------------------------
27
28  ;----- Memory Map WRAM ------------------------------------------------------
29  HOR_SPEED   = $0300     ; the horizontal speed
30  VER_SPEED   = $0301     ; the vertical speed
31  OAMMIRROR   = $0400     ; location of OAMRAM mirror in WRAM
32  ;------------------------------------------------------------------------------
33
34  ;----- Game Constants -------------------------------------------------------
35      ; we use these constants to check for collisions with the screen boundarie
```

```
36  SCREEN_LEFT     = $00   ; left screen boundary = 0
37  SCREEN_RIGHT    = $ff   ; right screen boundary = 255
38  SCREEN_TOP      = $00   ; top screen boundary = 0
39  SCREEN_BOTTOM   = $df   ; bottom screen boundary = 223
40      ; a simple constant to define the sprite movement speed
41  SPRITE_SPEED    = $02   ; the sprites will move 2 pixel per frame
42      ; this makes the code a bit more readable
43  SPRITE_SIZE     = $08   ; sprites are 8 by 8 pixel
44  OAMMIRROR_SIZE  = $0220 ; OAMRAM can hold data for 128 sprites, 4 bytes each
45  ;-------------------------------------------------------------------------
46
47  ;----- Assembler Directives ----------------------------------------------
48  .p816                           ; tell the assembler this is 65816 code
49  ;-------------------------------------------------------------------------
50
51  ;----- Includes ----------------------------------------------------------
52  .segment "SPRITEDATA"
53  SpriteData: .incbin "Sprites.vra"
54  ColorData:  .incbin "SpriteColors.pal"
55  ;-------------------------------------------------------------------------
56
57  .segment "CODE"
58  ;-------------------------------------------------------------------------
59  ;   This is the entry point of the demo
60  ;-------------------------------------------------------------------------
61  .proc   ResetHandler
62          sei                     ; disable interrupts
63          clc                     ; clear the carry flag
64          xce                     ; switch the 65816 to native (16-bit mode)
65          rep #$10                ; set X and Y to 16-bit
66          sep #$20                ; set A to 8-bit
67          lda #$8f                ; force v-blanking
68          sta INIDISP
69          stz NMITIMEN            ; disable NMI
70          ; set the stack pointer to $1fff
71          ldx #$1fff              ; load X with $1fff
72          txs                     ; copy X to stack pointer
73
74          ; load sprites into VRAM
75          tsx                     ; save current stack pointer
76          pea $0000               ; push VRAM destination address to stack
77          pea SpriteData          ; push sprite source address to stack
78          lda #$80                ; push number of bytes (128/$80) to transfer t
79          pha
```

```
80          jsr LoadVRAM            ; transfer VRAM data
81          txs                     ; "delete" data on stack by restoring old stac
82
83          ; load color data into CGRAM
84          tsx                     ; save current stack pointer
85          lda #$80                ; push CGRAM destination address to stack
86          pha
87          pea ColorData           ; push color source address to stack
88          lda #$20                ; push number of bytes (32/$20) to transfer to
89          pha
90          jsr LoadCGRAM           ; transfer color data into CGRAM
91          txs                     ; "delete" data on stack by restoring old stac
92
93          ; set up initial data in the OAMRAM mirror, use X as index
94          ldx #$00
95          ; upper-left sprite
96          lda #(SCREEN_RIGHT/2 - SPRITE_SIZE) ; sprite 1, horizontal position
97          sta OAMMIRROR, X
98          inx                                 ; increment index
99          lda #(SCREEN_BOTTOM/2 - SPRITE_SIZE); sprite 1, vertical position
100         sta OAMMIRROR, X
101         inx
102         lda #$00                            ; sprite 1, name
103         sta OAMMIRROR, X
104         inx
105         lda #$00                            ; no flip, palette 0
106         sta OAMMIRROR, X
107         inx
108         ; upper-right sprite
109         lda #(SCREEN_RIGHT/2)               ; sprite 3, horizontal position
110         sta OAMMIRROR, X
111         inx                                 ; increment index
112         lda #(SCREEN_BOTTOM/2 - SPRITE_SIZE); sprite 3, vertical position
113         sta OAMMIRROR, X
114         inx
115         lda #$01                            ; sprite 3, name
116         sta OAMMIRROR, X
117         inx
118         lda #$00                            ; no flip, palette 0
119         sta OAMMIRROR, X
120         inx
121         ; lower-left sprite
122         lda #(SCREEN_RIGHT/2 - SPRITE_SIZE) ; sprite 2, horizontal position
123         sta OAMMIRROR, X
```

```
124         inx                             ; increment index
125         lda #(SCREEN_BOTTOM/2)          ; sprite 2, vertical position
126         sta OAMMIRROR, X
127         inx
128         lda #$02                        ; sprite 2, name
129         sta OAMMIRROR, X
130         inx
131         lda #$00                        ; no flip, palette 0
132         sta OAMMIRROR, X
133         inx
134         ; lower-right sprite
135         lda #(SCREEN_RIGHT/2)            ; sprite 4, horizontal position
136         sta OAMMIRROR, X
137         inx                             ; increment index
138         lda #(SCREEN_BOTTOM/2)          ; sprite 4, vertical position
139         sta OAMMIRROR, X
140         inx
141         lda #$03                        ; sprite 4, name
142         sta OAMMIRROR, X
143         inx
144         lda #$00                        ; no flip, palette 0
145         sta OAMMIRROR, X
146         inx
147         ; move the other sprites off screen
148         lda #$ff                ; set the coordinates to (255, 255), which is
149 OAMLoop:
150         sta OAMMIRROR, X
151         inx
152         cpx #OAMMIRROR_SIZE
153         bne OAMLoop
154
155         ; correct extra OAM byte for first four sprites
156         ldx #$0200
157         lda #$00
158         sta OAMMIRROR, X
159
160         ; set initial horizontal and vertical speed
161         lda #SPRITE_SPEED
162         sta HOR_SPEED
163         sta VER_SPEED
164
165         ; .byte $42, $00          ; debugger breakpoint
166
167         ; make Objects visible
```

```
168         lda #$10
169         sta TM
170         ; release forced blanking, set screen to full brightness
171         lda #$0f
172         sta INIDISP
173         ; enable NMI, turn on automatic joypad polling
174         lda #$81
175         sta NMITIMEN
176
177         jmp GameLoop            ; all initialization is done
178 .endproc
179 ;-------------------------------------------------------------------------
180
181 ;-------------------------------------------------------------------------
182 ;   After the ResetHandler will jump to here
183 ;-------------------------------------------------------------------------
184 ; .smart ; keep track of registers widths
185 .proc   GameLoop
186         wai                     ; wait for NMI / V-Blank
187
188         .byte $42, $00          ; debugger breakpoint
189         ; Game Logic: Move the sprites
190         ; move sprite 1 horizontally
191         ; check collision left boundary
192         lda HOR_SPEED
193         bpl RightBoundaryCheck  ; if sprites are moving right, skip left bound
194         lda OAMMIRROR
195         clc
196         adc HOR_SPEED
197         bcs UpdateHorPosition
198             ; else, reposition sprite 1 to horizontal position to zero
199             stz OAMMIRROR           ; reposition sprite 1
200             bra InvertHorSpeed      ; invert the horizontal speed
201         ; check right boundary
202 RightBoundaryCheck:
203         lda OAMMIRROR
204         clc
205         adc HOR_SPEED
206         cmp #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
207         bcc UpdateHorPosition   ; if sprite 1 is two sprites-wide to the left
208             ; else, reposition sprite 1 horizontally
209             lda #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
210             sta OAMMIRROR           ; reposition sprite 1 horizontally
211             bra InvertHorSpeed      ; invert the horizontal speed
```

```
212 UpdateHorPosition:
213         sta OAMMIRROR           ; if no collision happened, store new position
214         bra VerticalCheck       ; check vertical collision
215         ; invert the horizontal speed after bouncing of the left/right screen
216 InvertHorSpeed:
217         lda HOR_SPEED           ; load current horizontal speed
218         eor #$ff                ; flip all bits
219         clc                     ; add 1 to inverted speed
220         adc #$01
221         sta HOR_SPEED           ; store inverted speed
222
223         ; move sprite 1 vertically
224 VerticalCheck:
225         ; check collision upper boundary
226         lda VER_SPEED
227         bpl CheckLowerBoundary  ; if sprites are moving down, skip upper bound
228         lda OAMMIRROR + $01
229         clc
230         adc VER_SPEED
231         bcs UpdateVerPosition
232             ; else, reposition sprite 1 to horizontal position to zero
233             stz OAMMIRROR + $01     ; reposition sprite 1
234             bra InvertVerSpeed      ; invert the horizontal speed
235         ; check lower boundary
236 CheckLowerBoundary:
237         lda OAMMIRROR + $01
238         clc
239         adc VER_SPEED
240         cmp #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
241         bcc UpdateVerPosition
242             ; else, reposition sprite 1 horizontally
243             lda #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
244             sta OAMMIRROR + $01     ; reposition sprite 1 horizontally
245             bra InvertVerSpeed      ; invert the horizontal speed
246 UpdateVerPosition:
247         sta OAMMIRROR + $01     ; if no collision happened, store new position
248         bra UpdateOtherSprites  ; check vertical collision
249         ; invert the horizontal speed after bouncing of the left/right screen
250 InvertVerSpeed:
251         lda VER_SPEED           ; load current horizontal speed
252         eor #$ff                ; flip all bits
253         clc                     ; add 1 to inverted speed
254         adc #$01
255         sta VER_SPEED           ; store inverted speed
```

```
256
257 UpdateOtherSprites:
258         lda OAMMIRROR           ; get new horizontal position of sprite 1
259         sta OAMMIRROR + $08     ; update sprite 3
260         clc
261         adc #SPRITE_SIZE
262         sta OAMMIRROR + $04     ; update sprite 2
263         sta OAMMIRROR + $0c     ; update sprite 4
264         ; vertical position
265         lda OAMMIRROR + $01     ; get new horizontal position of sprite 1
266         sta OAMMIRROR + $05     ; update sprite 2
267         clc
268         adc #SPRITE_SIZE
269         sta OAMMIRROR + $09     ; update sprite 3
270         sta OAMMIRROR + $0d     ; update sprite 4
271
272         jmp GameLoop
273 .endproc
274 ;-------------------------------------------------------------------------------
275
276 ;-------------------------------------------------------------------------------
277 ;   Will be called during V-Blank every frame
278 ;-------------------------------------------------------------------------------
279 .proc   NMIHandler
280         lda RDNMI               ; read NMI status, acknowledge NMI
281
282         ; this is where we would do graphics update
283         tsx                     ; save old stack pointer
284         pea OAMMIRROR           ; push mirror address to stack
285         jsr UpdateOAMRAM        ; update OAMRAM
286         txs                     ; restore old stack pointer
287
288         rti
289 .endproc
290 ;-------------------------------------------------------------------------------
291
292 ;-------------------------------------------------------------------------------
293 ;   Is not used in this program
294 ;-------------------------------------------------------------------------------
295 .proc   IRQHandler
296         ; code
297         rti
298 .endproc
299 ;-------------------------------------------------------------------------------
```

```
300
301 ;---------------------------------------------------------------------------
302 ;   Load sprite data into VRAM
303 ;   Parameters: NumBytes: .byte, SrcPointer: .addr, DestPointer: .addr
304 ;---------------------------------------------------------------------------
305 .proc   LoadVRAM
306         phx                     ; save old stack pointer
307         ; create frame pointer
308         phd                     ; push Direct Register to stack
309         tsc                     ; transfer Stack to... (via Accumulator)
310         tcd                     ; ...Direct Register.
311         ; use constants to access arguments on stack with Direct Addressing
312         NumBytes    = $07       ; number of bytes to transfer
313         SrcPointer  = $08       ; source address of sprite data
314         DestPointer = $0a       ; destination address in VRAM
315
316         ; set destination address in VRAM, and address increment after writing
317         ldx DestPointer         ; load the destination pointer...
318         stx VMADDL              ; ...and set VRAM address register to it
319         lda #$80
320         sta VMAINC              ; increment VRAM address by 1 when writing to
321
322         ; loop through source data and transfer to VRAM
323         ldy #$0000              ; set register Y to zero, we will use Y as a l
324 VRAMLoop:
325         lda (SrcPointer, S), Y  ; get bitplane 0/2 byte from the sprite data
326         sta VMDATAL             ; write the byte in A to VRAM
327         iny                     ; increment counter/offset
328         lda (SrcPointer, S), Y  ; get bitplane 1/3 byte from the sprite data
329         sta VMDATAH             ; write the byte in A to VRAM
330         iny                     ; increment counter/offset
331         cpy NumBytes            ; check whether we have written $04 * $20 = $8
332         bcc VRAMLoop            ; if X is smaller than $80, continue the loop
333
334         ; all done
335         pld                     ; restore caller's frame pointer
336         plx                     ; restore old stack pointer
337         rts
338 .endproc
339 ;---------------------------------------------------------------------------
340
341 ;---------------------------------------------------------------------------
342 ;   Load color data into CGRAM
343 ;   NumBytes: .byte, SrcPointer: .byte, DestPointer: .addr
```

```
344 ;------------------------------------------------------------------------
345 .proc   LoadCGRAM
346         phx                     ; save old stack pointer
347         ; create frame pointer
348         phd                     ; push Direct Register to stack
349         tsc                     ; transfer Stack to... (via Accumulator)
350         tcd                     ; ...Direct Register.
351         ; use constants to access arguments on stack with Direct Addressing
352         NumBytes    = $07       ; number of bytes to transfer
353         SrcPointer  = $08       ; source address of sprite data
354         DestPointer = $0a       ; destination address in VRAM
355
356         ; set CGDRAM destination address
357         lda DestPointer         ; get destination address
358         sta CGADD               ; set CGRAM destination address
359
360         ldy #$0000              ; set Y to zero, use it as loop counter and of
361 CGRAMLoop:
362         lda (SrcPointer, S), Y  ; get the color low byte
363         sta CGDATA              ; store it in CGRAM
364         iny                     ; increase counter/offset
365         lda (SrcPointer, S), Y  ; get the color high byte
366         sta CGDATA              ; store it in CGRAM
367         iny                     ; increase counter/offset
368         cpy NumBytes            ; check whether 32/$20 bytes were transfered
369         bcc CGRAMLoop           ; if not, continue loop
370
371         ; all done
372         pld                     ; restore caller's frame pointer
373         plx                     ; restore old stack pointer
374         rts
375 .endproc
376 ;------------------------------------------------------------------------
377
378 ;------------------------------------------------------------------------
379 ;   Copies the OAMRAM mirror into OAMRAM
380 ;------------------------------------------------------------------------
381 .proc   UpdateOAMRAM
382         phx                     ; save old stack pointer
383         ; create frame pointer
384         phd                     ; push Direct Register to stack
385         tsc                     ; transfer Stack to... (via Accumulator)
386         tcd                     ; ...Direct Register.
387         ; use constants to access arguments on stack with Direct Addressing
```

```
388         MirrorAddr  = $07        ; address of the mirror we want to copy
389
390         ; set up DMA channel 0 to transfer data to OAMRAM
391         lda #%00000010          ; set DMA channel 0
392         sta DMAP0
393         lda #$04                ; set destination to OAMDATA
394         sta BBAD0
395         ldx MirrorAddr          ; get address of OAMRAM mirror
396         stx A1T0L               ; set low and high byte of address
397         stz A1T0B               ; set bank to zero, since the mirror is in WRA
398         ldx #$0220              ; set the number of bytes to transfer
399         stx DAS0L
400
401         lda #$01                ; start DMA transfer
402         sta MDMAEN
403
404         ; OAMRAM update is done, restore frame and stack pointer
405         pld                     ; restore caller's frame pointer
406         plx                     ; restore old stack pointer
407         rts
408 .endproc
409 ;-------------------------------------------------------------------------------
410
411 ;-------------------------------------------------------------------------------
412 ;   Interrupt and Reset vectors for the 65816 CPU
413 ;-------------------------------------------------------------------------------
414 .segment "VECTOR"
415 ; native mode   COP,        BRK,        ABT,
416 .addr           $0000,      $0000,      $0000
417 ;               NMI,        RST,        IRQ
418 .addr           NMIHandler, $0000,      $0000
419
420 .word           $0000, $0000    ; four unused bytes
421
422 ; emulation m.  COP,        BRK,        ABT,
423 .addr           $0000,      $0000,      $0000
424 ;               NMI,        RST,        IRQ
425 .addr           $0000,      ResetHandler, $0000
```

BouncingSprite.asm hosted with ❤ by **GitHub** **view raw**

Lines 17 through 25: These are some new labels we'll use for setting up and using Direct Memory Access.

Lines 28 through 32: We'll use this memory location to store the horizontal and vertical speed of the sprites, and the (starting) address of the OAMRAM mirror in WRAM. Remember that we can access parts of the SNES' WRAM from the addresses `$00:0000 ~ $00:1fff`.

Lines 34 through 45: These are a bunch of constants to make our code more readable. We'll see them in action shortly.

Lines 61 through 91: Nothing new here. We initialize the SNES and copy the sprite and color data into VRAM and CGRAM, respectively.

Lines 93 through 158: This section is new. Basically, we write all data needed to display the four sprites into WRAM. We use the constants we defined earlier to position all four sprites at the center of the screen. Then, in `OAMLoop`, we move the other 124 sprites off screen, so they don't interfere with the sprites on screen.

If this section is a bit confusing, revisit Part 4 where we talked about sprites and what data is needed to display them on screen. I'll also link some extra information in the *Links and References* section at the end of this article.

Lines 160 through 163: We initialize the (initial) horizontal and vertical speed of the sprites. Mind that we could use constants only for this throughout the code. I chose to use "variables" (i.e., a memory location) to make the code more flexible and easier for you to experiment with.

Now, let's get to the real juicy bits.

Lines 185 through 269: This is our main game loop. Here is the new code that will move the sprites each frame according to the speeds we just stored in `HOR_SPEED` and `VER_SPEED`. This code has two sections: First, we check for horizontal (i.e., the left and right screen boundaries) collisions, then we check for vertical collisions (i.e., top and bottom of the screen). Please keep in mind that this main game loop is not optimized; there's a lot of code repetition and redundancies. My goal was to write the code as logically and as simple as possible to follow. As you gain experience, you'll be able to cut down code size and cycle count easily on this game logic.

First, in lines 192 and 193 we check whether the current speed is positive/to the right, then we skip the check on the check for the left screen boundary (the sprite can't hit the left screen boundary while it is moving right). BPL is a new instruction:

```
bpl : Branch if PLus
```

BPL will branch if the result of the last operation is positive; i.e., if the most significant bit of the result (of the last operation) is clear.

If the horizontal speed is *not* positive (BPL didn't not branch), we calculate the new position and check it against the left screen boundary.

Lines 195, 196, and 197 take the horizontal position of the first sprite and add the speed to calculate the new position. Then we the carry flag to check whether the value crossed the zero boundary (i.e, went below zero). If you're unsure why and how this works, freshen up your knowledge on two-complements binary number arithmetics. If the new position is below zero, BCS in line 197, the sprite moved beyond the left screen boundary; so we reset the horizontal sprite position to zero and branch to a section of code to invert the speed.

Lines 203 through 211 are pretty much a mirror of the left screen boundary check directly above it, except we reset the position to sprites to the left of the right screen boundary.

Lines 213 and 214 are the "standard case": If there was neither a collision with the left or right screen boundary, we simply store the new horizontal position in the OAMRAM mirror and branch to the vertical collision check.

Lines 216 through 221 are only executed if there was a collision with the left or right screen boundary before. We load the current speed, invert all bits, and add one to it; better known as the two-complements of a number (i.e., we change the sign/direction of the speed).

Lines 224 through 255 are again a mirror of the horizontal check for the vertical collision of the upper and bottom screen boundaries. If you carefully studied the horizontal collision code, this section will hold no surprises for you.

Lines 257 through 270: Now that we've calculated the correct new position of the first of the four sprites, we need to update the other three. Since we know how the other three sprites are positioned relative to the first, we only need to add the sprite size to the new horizontal and vertical position of the first sprite and store it as position data for the other sprites. We know that each sprite needs four bytes of data in OAMRAM, so the offset for accessing the data of the other sprites is four.



Relative position of sprites

If you feel like this is a very cumbersome way to update sprites that always move together, you're correct. In a later article, I'll introduce you to metasprites. Metasprites will make it easier to manage game characters, bosses, etc. made up of several sprites and are very common among 8- and 16-bit games.

I'd again like to stress that this game logic code has a lot of code repetition and potential for optimization. But I wanted to keep it simple to follow and understand.

Keen observers will have noticed that I didn't use two constants at all, namely, `SCREEN_LEFT` and `SCREEN_TOP`. This is for you to experiment. Can you make the sprites bounce only on the right half of the screen? Or the lower half? Give it a try!

Lines 279 through 289: This is the updated subroutine `NMIHandler`. This subroutine is called *each frame during V-blanking*. We haven't talked about this in detail, but many of the memory mapped registers can only be used during V-blanking; i.e., you can't update graphics (VRAM, OAMRAM, etc.) while the PPU is drawing stuff on the screen.

The new code inside the `NMIHandler` shouldn't surprise you. We move the address of the OAMRAM mirror onto the stack (to pass it as an argument to the subroutine), then call the new subroutine `UpdateOAMRAM`, which we will discuss shortly.

Lines 295 through 375: Nothing new here. Check `SpriteDemo3.s` above for a detailed description.

Lines 381 through 408: Now, this is the most important new code. The subroutine `UpdateOAMRAM` uses *Direct Memory Access* to copy the OAMRAM mirror from WRAM into OAMRAM. Let's see how this works.

Lines 382 through 388 should be pretty familiar by now. Last time, we talked about frame pointers and how they work.

Lines 391 through 399 hold the important bits. If we want to move data from one memory segment to another, we need to know (or let the SNES know, to be precise) three things:

- *where* to move the data *to* (destination)
- *where* to take the data *from* (source)
- and *how many* bytes we want to move (size)

There are a total of seven DMA channels on the SNES. If you'd find yourself in a situation where you'd need to move several segments of memory of other segments of memory at the same time, that'd be possible. You'd just set up every channel individually and then start DMA. We're only moving data from WRAM to OAMRAM, so we'll only use DMA channel 0.

To provide the SNES with all necessary information, we again use memory mapped registers. First, we use `DMAP0` to configure DMA channel 0. Direct Memory Access is very flexible. I'll go into more detail in a later article about this; here we set the DMA to the simplest kind of transfer, copy one byte of data from the source (WRAM) and write it to a (memory mapped) register (i.e., the DMA destination), and increment the source address by one after each read-write-cycle. I'll provide some links in the *Links and References* section at the end of this article that explains this in more detail. We'll look more closely at this (and HDMA) in a later article, so trust me on this for now.

Next, we use `BBAD0` register to set the destination to `$04`. The destination of a DMA must be one of the memory mapped registers in the `$21XX` range (this is where all memory mapped registers concerning graphics are located). Since we want to move data to OAMRAM, we set it to `$04` which will result in a destination address/register of `$2104`; which is the OAMDATA register.

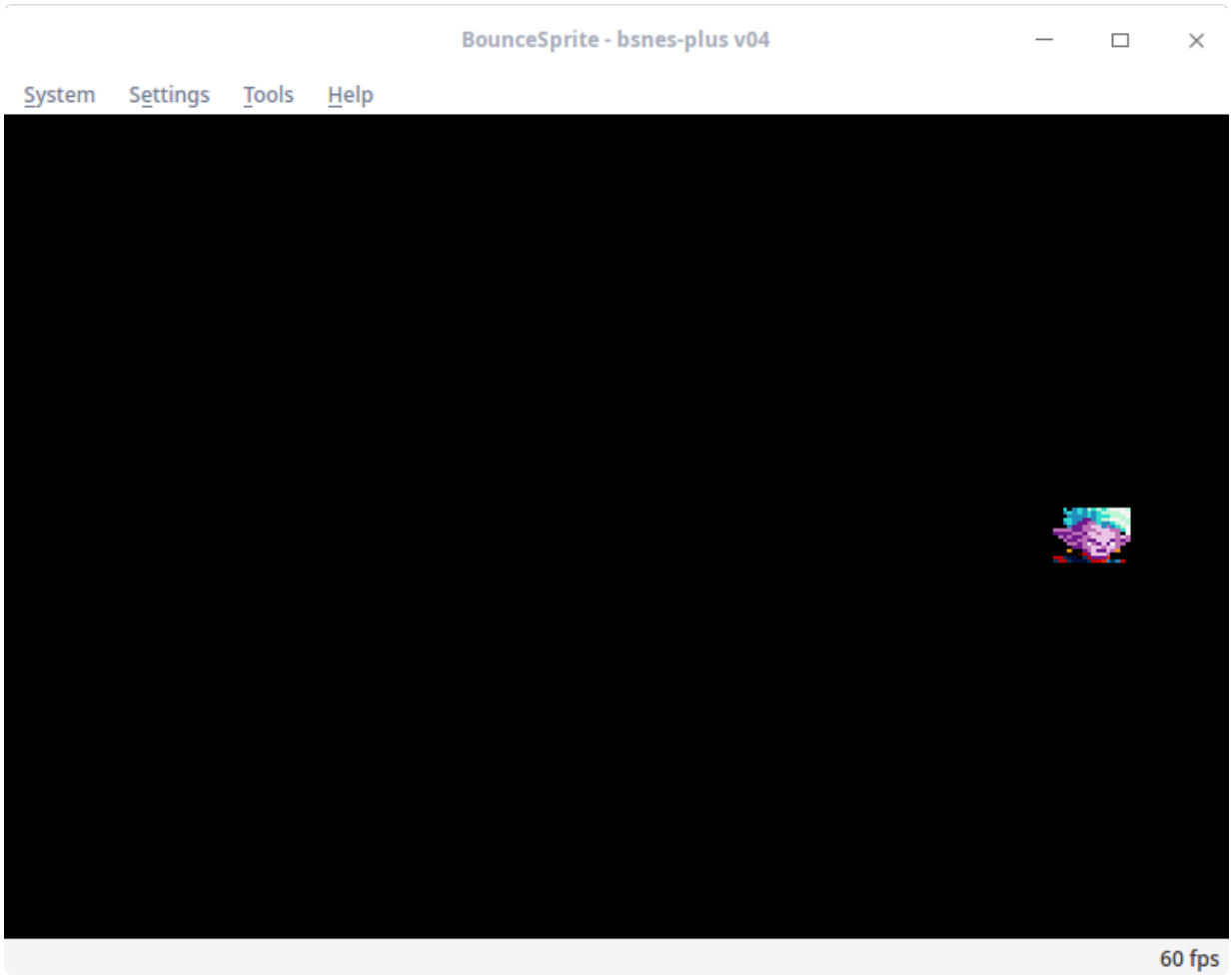
Then we use `A1T0L`, `A1T0H`, and `A1T0B` to set the destination address. We load the address of the OAMRAM mirror (earlier passed on stack) and set it as the source address. Since the use the WRAM mirror in bank 0, we use STZ to set the bank of the address to zero.

Lastly, we set the number of bytes to be transferred to `$0220`, which is the whole OAMRAM mirror (and the total size of OAMRAM).

Now that DMA is set up, we only need to tell the SNES to start the transfer. For this, we use the `MDMAEN` register to start the transfer on channel 0. If we wanted to start several DMA channels, we would set the appropriate bit in line 401 (bit 0 for channel 0, bit 1 for channel 1, etc.).

Once the DMA is done, we only need to restore the caller's frame and stack pointer, then we can return to the caller.

If you build this code and run it in your emulator, this is what you should see:



Bouncy McBounceface

Congratulations! Your first moving sprites!

Conclusion

There was a lot of new stuff here, so make sure to study the code examples closely. I'd also encourage you to experiment a bit. Can you make the sprites bounce only on the right half of

the screen? Can you modulize the game logic code with subroutines? Make each sprite move and bounce individually? Bounce off each other? Challenge yourself!

Next time, we'll learn how to read the joypad and make the sprites react to your input.

Lastly, I'm very sorry this update took so long; I've been working on a few other things but I'm determined to return to a weekly Thursday schedule as promised. Your feedback is much appreciated and a huge motivation! Stay tuned!

Links and References

- A good explanation of DMA from the Super Famicom Development Wiki
- A detailed explanation of OAMRAM from the same Wiki
- Retro Game Machines Explained has an excellent series of videos explaining how the SNES works, I can't recommend them enough!
- A short explanation of indirect and indexed addressing
- This post on NES Doug's excellent blog explains the basic idea behind metasprites
- All SNES Assembly Adventure code examples of this series on Github

ALSO ON MACHINE CODE CONSTRUCTION YARD

SNES Assembly Adventure 08: …

4 years ago · 1 comment

I do assembly programming on old machines and boat building.

SNES Assembly Adventure 09: Project …

4 years ago · 1 comment

I do assembly programming on old machines and boat building.

SNES Assembly Adventure 02: Basic …

8 years ago · 6 comments

I do assembly programming on old machines and boat building.

SNES Assem Adventure 0

8 years ago · 6 c

I do assembly on old machin building.